PriceOS

Internal Architecture & System Design

📄 Version 2.0 📅 March 6, 2026 🏗️ Production System

📋 Table of Contents

1. Executive Overview
2. Tech Stack
3. System Architecture
4. Database Schema
5. AI Agent Architecture
6. Safety & Anti-Hallucination
7. The 7-Step Intelligence Sweep
8. API Architecture
9. Frontend Architecture
10. Data Sync Pipeline
11. Security Architecture
12. Deployment & Cost
13. Key Design Decisions
14. Glossary
15. Detailed Agent Playbook

1

PriceOS is an **AI-powered dynamic pricing copilot** purpose-built for Dubai short-term rental property managers. It replaces manual pricing spreadsheets with an autonomous multi-agent system that scans the internet for market intelligence, benchmarks competitors in real-time, and generates daily pricing recommendations — all through a conversational interface called **Aria**.

💡 What Makes PriceOS Unique

Unlike traditional pricing tools that use static rules, PriceOS deploys 10 specialized AI agents that collaborate in real-time — scanning for geopolitical threats, competitor prices, events, and economic signals to generate defensible pricing decisions.

CAPABILITY	TRADITIONAL TOOLS	PRICEOS
Pricing decisions	Manual / rule-based	AI agents with real-time internet access
Market intelligence	Static quarterly reports	Live scanning (wars, events, currency, weather)

CAPABILITY	TRADITIONAL TOOLS	PRICEOS
User interface	Dashboards with sliders	Conversational AI ("Aria, what should I price?")
Competitor data	Manual surveys	Real-time Airbnb/Booking.com scraping
Safety guardrails	None	4-layer protection: PriceGuard + Guardrails Agent

2

LAYER	TECHNOLOGY	PURPOSE
Frontend	Next.js 16 + React 19	Server-rendered web application
Styling	Tailwind CSS 4 + shadcn/ui	Premium dark-mode UI
State	Zustand	Client-side state management
Database	Neon (Serverless PostgreSQL)	Auto-scaling Postgres
ORM	Drizzle ORM	Type-safe SQL, zero overhead
AI Platform	Lyzr Studio	Multi-agent orchestration
AI Models	Gemini 3.0 Flash + GPT-4o + GPT-4o-mini	Chat, search, compute
Internet Search	Perplexity Sonar LLM	Real-time web search
PMS	Hostaway API	Property & booking sync
Auth	Neon Auth (Stack Auth)	User management
Hosting	Vercel	Edge-optimized deployment

3.1 Two-Phase Architecture

PriceOS operates in **two distinct phases**. This is the most important architectural concept:

⚡ Phase 1: Setup

Trigger: User clicks "Run Aria"
Duration: 15-30 seconds
Agents: 6 (Marketing) + 7 (Benchmark) + 10 (Guardrails)
Internet: ✅ Yes — scans the web

💬 Phase 2: Chat

Trigger: User sends a message
Duration: 5-15 seconds
Agents: 1 (Aria) → 2, 3, 4, 5
Internet: ❌ No — uses cached data

🔑 Key Insight

Only Agents 6 and 7 have internet access. All chat-phase agents work with data cached from the last "Run Aria" scan. If an event is announced between scans, agents will be blind to it until the user triggers a new scan.

3.2 Data Flow

```
User clicks "Run Aria"
        │
        ├──→ Agent 6 (Marketing) ──🌐──→ Events, Wars, Weather ──→ market_events DB
        │
        ├──→ Agent 7 (Benchmark) ──🌐──→ Airbnb/Booking.com ──→ benchmark_data DB
        │
        └──→ Agent 10 (Guardrails) ──→ Floor/Ceiling prices ──→ listings DB

User sends message to Aria
        │
        ▼
    chat/route.ts ──→ Loads all cached data from DB
        │
        └──→ Injects JSON payload (today, metrics, events, benchmark)
             │
             ▼
        Agent 1 (Aria) ──→ Routes to sub-agents → Merges → Responds
             │
             ├──→ Agent 2 (Property Analyst) → Gaps, LOS, Seasonal
             ├──→ Agent 3 (Booking Intel) → Velocity, Revenue
             ├──→ Agent 4 (Market Research) → Events, News factors
```

```
                    └──→ Agent 5 (PriceGuard) → Final prices + validation
```

4

PriceOS uses **9 PostgreSQL tables** in Neon, designed for near-zero JSONB usage:

TABLE	ROLE	WRITTEN BY	READ BY
`listings`	Property registry	Hostaway Sync + Agent 10	All agents, UI
`inventory_master`	Daily price calendar	Hostaway Sync	Calendar UI, Chat
`reservations`	Guest bookings & revenue	Hostaway Sync	Dashboard, Agents
`market_events`	AI market intelligence	Agent 6 (Marketing)	Agent 4, Chat payload
`benchmark_data`	Competitor pricing	Agent 7 (Benchmark)	Agent 5, Chat payload
`chat_messages`	Conversation history	Chat API	Chat UI
`user_settings`	User configuration	Settings UI	Auth, Sync
`guest_summaries`	AI guest comms analysis	Summary Agent	Inbox UI
`hostaway_conversations`	Raw guest messages	Hostaway Sync	Guest Reply Agent

AGENT 1

CRO Router ("Aria")

Gemini 3.0 Flash · CHAT

User-facing orchestrator. Routes queries, merges outputs, delivers reports.

AGENT 2

Property Analyst

GPT-4o-mini · CHAT

Gap detection, LOS optimization, seasonal patterns, revenue forecasts.

AGENT 3

Booking Intelligence

GPT-4o-mini · CHAT

Velocity analysis, revenue breakdown, channel mix, booking patterns.

AGENT 4

Market Research

GPT-4o-mini · CHAT

Parses pre-cached events, news, competitors into pricing factors.

AGENT 5

PriceGuard

GPT-4o-mini · VALIDATOR

Final pricing engine with veto power. Crisis override capability.

AGENT 6

Marketing Intelligence

GPT-4o (Sonar) · INTERNET

7-step internet sweep: threats, events, economy, weather, viral signals.

AGENT 7

Benchmark Agent

GPT-4o (Sonar) · INTERNET

Competitor pricing from Airbnb/Booking.com with scale validation.

AGENT 10

Guardrails Agent

GPT-4o-mini · SETUP

Computes intelligent floor/ceiling prices from benchmark data.

UTILITY

Conv. Summary Agent

GPT-4o-mini · ON-DEMAND

Analyzes guest conversations for patterns, themes, action items.

UTILITY

Guest Reply Agent

GPT-4o-mini · ON-DEMAND

Auto-generates warm, accurate guest replies with DB lookup.

6

PriceOS implements **4 layers of protection** against AI errors:

LAYER	AGENT	PROTECTION	EXAMPLE
LAYER 1	Agent 6	"2026 Mention Rule" — rejects events without explicit year on source	Prevents 2025 Art Dubai dates being used for 2026
LAYER 2	Agent 7	Bedroom-aware rate thresholds (1BR <1500, 4BR <6000 AED)	Rejects monthly rental data mistaken as nightly
LAYER 3	Agent 5	Geopolitical crisis override — wars cancel event premiums	Iran tensions → drop to P25 pricing for occupancy
LAYER 4	Agent 1	Anomaly alert — warns user if market data seems wrong	"Market median seems wrong — using conservative pricing"

7

Agent 6 (Marketing) executes this comprehensive sweep every time "Run Aria" is triggered:

1. 🔴 **Geopolitical Threat Scan** — Travel advisories, Iran-UAE tensions, Yemen Houthi attacks, flight disruptions, pandemics. Impact: -40% to -60%.
2. 📅 **Calendar & Religious Dates** — Ramadan phase (early/mid/late/Eid), UAE holidays, school holidays. Impact: -20% to +50%.
3. 🎪 **Major Events** — Exhibitions, conferences, festivals, concerts, sports in date range. Impact: +10% to +30%.
4. 🏘️ **Neighborhood Intelligence** — Landmark closures, local activities, micro-events. Impact: -10%.
5. 💹 **Economic & Currency Signals** — INR/GBP/EUR exchange rates, oil prices, hotel occupancy, tourism stats. Impact: -3% to +5%.
6. 🌡️ **Weather & Environment** — Heat waves, sandstorms, pleasant conditions. Impact: -40% (summer).
7. 📱 **Viral & Trending** — Dubai trending on social media, scam warnings, new attractions. Impact: ±5%.

ROUTE	METHOD	PURPOSE
`/api/chat`	POST	Send message to Aria, get AI response
`/api/market-setup`	POST	Trigger Setup Phase (Agents 6, 7, 10)
`/api/calendar-metrics`	GET	Occupancy, booked/available/blocked days
`/api/calendar`	GET	Daily inventory with prices
`/api/listings`	GET	Property list with details
`/api/benchmark`	GET	Competitor benchmark data
`/api/events`	GET	Market events and news
`/api/reservations`	GET	Guest bookings
`/api/proposals`	GET/POST	AI pricing proposals
`/api/sync/[type]`	POST	Hostaway data sync
`/api/conversations`	GET	Guest communication threads

External Integrations

SERVICE	PURPOSE	METHOD
Lyzr Studio	AI agent hosting & inference	REST API
Hostaway	PMS data sync	REST API (100 req/min)
Neon	PostgreSQL database	Serverless WebSocket
Perplexity Sonar	Internet search	Via Lyzr agents

Page Map

ROUTE	PAGE	KEY COMPONENTS
`/dashboard`	Overview	MetricCards, OccupancyChart, RevenueChart
`/dashboard/agent-chat`	Aria Chat	ChatPanel, MessageBubble, ProposalDrawer
`/dashboard/calendar`	Calendar	CalendarGrid, DayCell, PriceEditor
`/dashboard/properties`	Properties	PropertyCard, SyncStatusBar
`/dashboard/bookings`	Bookings	BookingTable, ReservationDetail
`/dashboard/inbox`	Guest Inbox	ConversationList, MessageThread
`/dashboard/finance`	Finance	RevenueChart, ExpenseTable
`/dashboard/insights`	Intelligence	EventTimeline, BenchmarkChart
`/dashboard/pricing`	Pricing	PriceTable, ProposalReview

10

```
Hostaway PMS API
        │
        ▼
┌──────────────────┐
│  /api/sync/[type] │    Types: listings, calendar, reservations, conversations
│  Sync Controller  │
└──────────────────┘
            │
            ▼
    1. Fetch page-by-page from Hostaway API
    2. Transform to PriceOS schema
    3. Upsert (INSERT ... ON CONFLICT UPDATE)
    4. Track sync metadata (last_synced)
    5. Handle rate limits (100 req/min)
            │
            ▼
┌──────────────────────────────────────────┐
│  Neon PostgreSQL                          │
│  listings │ inventory_master │ reservations │
└──────────────────────────────────────────┘
```

11

LAYER	IMPLEMENTATION
Authentication	Neon Auth (Stack Auth) — JWT sessions
API Protection	Every route validates userId from session
Data Isolation	All queries filtered by userId — no cross-user leakage
AI Safety	PriceGuard has unconditional veto power
Price Guardrails	Floor/ceiling prevent catastrophic pricing
Guest Privacy	AI never exposes PII outside owner context

12

```
┌──────────────────────┐      ┌──────────────────────┐
│   Vercel Edge        │      │   Neon Serverless    │
│   (Next.js SSR)      │──────→│   (PostgreSQL)       │
│   Auto-scaling       │      │   Auto-scaling       │
└──────────────────────┘      └──────────────────────┘
          │
          ├──→ Lyzr Studio (AI inference)
          ├──→ Hostaway API (PMS data)
          └──→ Perplexity Sonar (web search)
```

SERVICE	APPROX. MONTHLY (10 PROPERTIES)
Vercel	~$20/mo
Neon PostgreSQL	~$19/mo
Lyzr Studio	~$50-100/mo
Hostaway API	Included with PMS

13

Why Multi-Agent (Not Single LLM)?

A single LLM cannot search the internet, validate prices, AND generate reports in one call. 10 specialized agents each have a narrow, verifiable scope — if PriceGuard rejects a price, we know exactly why.

Why Two-Phase (Not Real-Time)?

Internet searches take 15-30 seconds — too slow for real-time. Competitor prices don't change minute-by-minute. Property managers want to review pricing before applying.

Why Floor/Ceiling Guardrails?

Pricing errors are asymmetric: overpricing loses one night; underpricing creates a viral "cheap" perception that's hard to undo. Guardrails create a safe zone for AI decisions.

TERM	DEFINITION
PMS	Property Management System (Hostaway) — source of truth for bookings
STR	Short-Term Rental (Airbnb, Booking.com listings)
ADR	Average Daily Rate — total revenue ÷ booked nights
LOS	Length of Stay — min/max night requirements
P50 / P75 / P90	Percentile rates from competitor benchmark
CRO	Chief Revenue Officer — the persona Aria embodies
Sonar LLM	Perplexity's internet-connected language model
Setup Phase	Internet-connected scan triggered by "Run Aria"
Chat Phase	Conversational AI using pre-cached data
PriceGuard	Agent 5 — the final pricing validator with veto power

This section provides a deep-dive into the logic and output format of each core agent in the PriceOS ecosystem.

AGENT 1 CRO Router (Aria)

Responsibility: The "Brain" of PriceOS. Aria is an orchestrator that translates loose human intentions into structured multi-agent tasks.

How it Works:

1. Parses the user message (e.g., "Full analysis for next week").
2. Constructs the `System Payload` (metrics, bookings, market data).
3. Parallel-executes **Agents 2, 3, 4, 5**.
4. Merges JSON results into a unified XML-structured report.

Sample Output (Internal Report State):

```
<analysis_report>
  <executive_summary>🔴 Red Alert: Airspace closure NOTAM issued...</executive_summa
  <opportunity_score>85/100</opportunity_score>
  <sub_reports>
    <property_logic>Gap nights detected on Mar 12-14...</property_logic>>
    <price_results>[14 proposals generated]</price_results>
  </sub_reports>
</analysis_report>
```

AGENT 2 Property Analyst

Responsibility: Maximizing occupancy through gap detection and inventory management.

How it Works:

Scans the `inventory` array day-by-day. If it finds isolated available nights between bookings (e.g., a 2-night gap between two 5-night bookings), it flags them for special pricing or reduced Min Stay logic.

Sample Output:

```
{
  "gaps": [{"start": "2026-03-12", "end": "2026-03-14", "nights": 2}],
  "restrictions": "Reduce Min Stay from 3 to 2 for Mar 12 gap",
```

```
  "seasonal_analysis": "Entry into Ramadan shoulder period: -10% ADR expected."
}
```

AGENT 5 PriceGuard

Responsibility: The Final Validator. It holds "Veto Power" over all pricing decisions.

How it Works:

1. Receives suggested premiums from Market Research (Agent 4).
2. Computes `Final_Price = (Base_Rate * Trend_Multiplier) + Context_Premiums`.
3. Force-clamps the result to the `floor_price` and `ceiling_price`.
4. Validates the price against `Agent 7's` P50/P90 benchmarks.

Sample Output (Individual Proposal):

```
{
  "date": "2026-03-15",
  "proposedPrice": 850,
  "reasoning": {
    "market": "P75 position maintained despite Ramadan mid-phase",
    "occupancy": "Property at 0% occupancy for window; price lowered to floor",
    "threats": "Geopolitical risk level 3 active; -15% safety buffer applied"
  }
}
```

AGENT 6 Marketing Intelligence

Responsibility: Real-time internet scanning with a "Skepticism Layer."

How it Works:

Uses **Perplexity Sonar** to execute a 7-step sweep. Every "alert" found is fact-checked: Does it mention 2026? Is there a second official source? This prevents the system from over-reacting to fake or outdated news.

Sample Output:

```
{
  "news": [{
    "headline": "Dubai Airspace Remains Open (Official WAM)",
    "category": "security",
```

```
      "sentiment": "positive",
      "confidence": 98,
      "description": "Cross-verified with Emirates and DXB Airport arrivals."
    }],
    "demand_outlook": {"trend": "strong", "reason": "Art Dubai surge"}
  }
```

AGENT 3 Booking Intelligence

Responsibility: Analyzing internal booking "pacing" and velocity to detect demand strength before the market does.

How it Works:

Compares `recent_reservations` against historical averages for the same period. If booking lead time is shrinking, it flags a "High Velocity" signal to raise prices.

Sample Output:

```
{
  "velocity_score": 8.5,
  "pacing_verdict": "Ahead of target (+12% YoY)",
  "channel_mix": "Direct: 15%, Airbnb: 70%, Booking: 15%"
}
```

AGENT 4 Market Research

Responsibility: Translating raw external events into numerical price multipliers.

How it Works:

Reviews the `market_events` and `news` tables. It assigns an `impact_multiplier` to each date based on proximity to events like Art Dubai or COP28.

Sample Output:

```
{
  "event_scores": [{"event": "Art Dubai", "score": 1.25, "dates": ["2026-03-06", "20
  "sentiment_multiplier": 0.95
}
```

AGENT 7 Benchmark Agent

Responsibility: Continuous competitor "scraping" and price normalization.

How it Works:

Executes live searches for "Dubai [Area] 1BR Apartment" on Airbnb/Booking. It extracts the **exact nightly rate** for the next 14 days, excluding cleaning fees, to ensure an apples-to-apples comparison.

Sample Output:

```
{
  "verdict": "Priced Aggressively (P90)",
  "p50_median": 650,
  "p90_high": 1200,
  "comps": [{"name": "Palm Villa 4", "price": 1150, "rating": 4.9, "reviews": 122}]
}
```

AGENT 10 Guardrails Agent

Responsibility: Establishing the "Safe Zone" for AI operations.

How it Works:

Runs only during the **Setup Phase**. It looks at the property's fixed costs and the Market Median (P50) to set a **Floor** below which the AI cannot go, and a **Ceiling** that prevents price gouging.

Sample Output:

```
{
  "new_floor_price": 450,
  "new_ceiling_price": 1850,
  "reasoning": "Floor set at 15% below P50 to ensure high occupancy in low season."
}
```